I. Description of Overall Test Plan

Our main testing strategy for Code Colosseum is developer-driven testing. Hence, whenever a contributor implements a new feature, they are also responsible for testing its functionality. This has worked mainly as our project is small in scope. However, as the project scales, we execute more comprehensive test suites to ensure that core features remain stable.

To mitigate the potential bias in developer testing, we supplement this with external validation. We have released the project to a select focus group that is familiar with the application's intended purpose. This group validates core functionality to ensure the system behaves as expected from an end-user perspective.

II. Test Case Descriptions

TC-01

- **Purpose:** Verify home page renders
- **Description:** Render the landing page and confirm primary UI text is present.
- **Inputs:** Component: `Home`
- **Outputs:** “To be Implemented”, “Practice”, and “Competitive” are visible.
- **Type:** Normal
- **Method:** Blackbox
- **Category:** Functional
- **Level:** Integration
- **Results:** The Home component rendered successfully; all placeholder text and navigation options were visible.

TC-02

- **Purpose:** Validate the login page loads
- **Description:** Render the login page and verify the GitHub login CTA.
- **Inputs:** Component: `LoginPage`
- **Outputs:** “Welcome to the Arena” and “Sign in with GitHub” are visible.
- **Type:** Normal
- **Method:** Blackbox
- **Category:** Functional
- **Level:** Integration
- **Results:** Login page loaded correctly; GitHub authentication button was present and interactive.

TC-03

- **Purpose:** Competitive page renders
- **Description:** Render a competitive page and verify the leaderboard section.
- **Inputs:** Component: `CompetitivePage`
- **Outputs:** “Colosseum”, “Global Rankings”, and leaderboard entries appear.
- **Type:** Normal
- **Method:** Blackbox
- **Category:** Functional
- **Level:** Integration
- **Results:** Page rendered with "Colosseum" header; global ranking table populated with mock entries.

TC-04

- **Purpose:** Practice list renders
- **Description:** Render the practice list and verify the question list after loading.
- **Inputs:** Component: `PracticeListPage` with mocked data
- **Outputs:** Question titles appear after the loading state.
- **Type:** Normal
- **Method:** Blackbox
- **Category:** Functional
- **Level:** Integration
- **Results:** Loading spinner appeared briefly, followed by the correct list of question titles.

TC-05

- **Purpose:** Practice list filters
- **Description:** Click a category and verify the list filters for that category.
- **Inputs:** Component: `PracticeListPage` with mocked data
- **Outputs:** Only category-matching questions remain.
- **Type:** Normal
- **Method:** Blackbox
- **Category:** Functional
- **Level:** Integration
- **Results:** Selecting the "Arrays" filter removed all non-Array questions from the view.

TC-06

- **Purpose:** Question data loader
- **Description:** Validate the question retrieval service returns the expected data shape.
- **Inputs:** Call `getQuestions()` in `src/services/questions.ts`
- **Outputs:** Array of questions with required fields (id, title, difficulty, content).
- **Type:** Normal
- **Method:** Whitebox
- **Category:** Functional
- **Level:** Unit

- **Results:** Service returned a valid JSON array; all objects contained the required schema keys.

TC-07

- **Purpose:** Question by id
- **Description:** Validate question retrieval by id.
- **Inputs:** Call `getQuestionById()` in `src/services/questions.ts`
- **Outputs:** Returns a single question object.
- **Type:** Normal
- **Method:** Whitebox
- **Category:** Functional
- **Level:** Unit
- **Results:** Querying ID '1' returned the specific object for 'Two Sum'.

TC-08

- **Purpose:** Available languages error
- **Description:** Validate error handling in the available languages fetch.
- **Inputs:** Call `getAvailableLanguages()` with mocked error
- **Outputs:** Returns an empty array.
- **Type:** Abnormal
- **Method:** Whitebox
- **Category:** Functional
- **Level:** Unit
- **Results:** Service caught the mock error and defaulted to returning `[]` without crashing.

TC-09

- **Purpose:** Execute API unsupported language
- **Description:** Submit unsupported language to execute the API.
- **Inputs:** POST `/api/execute` with invalid `language`
- **Outputs:** 400 responses with “Unsupported Language”.
- **Type:** Abnormal
- **Method:** Blackbox
- **Category:** Functional
- **Level:** Integration
- **Results:** API returned HTTP 400; body contained correct error message.

TC-10

- **Purpose:** Execute API success
- **Description:** Submit a valid execution request with mocked dependencies.
- **Inputs:** POST `/api/execute` with a valid payload
- **Outputs:** 200 responses with Accepted status and stdout.

- **Type:** Normal
- **Method:** Whitebox
- **Category:** Functional
- **Level:** Integration

Test Case ID	Normal/Abnormal	Blackbox/Whitebox	Functional/Performance	Unit/Integration
TC-01	Normal	Blackbox	Functional	Integration
TC-02	Normal	Blackbox	Functional	Integration
TC-03	Normal	Blackbox	Functional	Integration
TC-04	Normal	Blackbox	Functional	Integration
TC-05	Normal	Blackbox	Functional	Integration
TC-06	Normal	Whitebox	Functional	Unit
TC-07	Normal	Whitebox	Functional	Unit
TC-08	Abnormal	Whitebox	Functional	Unit
TC-09	Abnormal	Blackbox	Functional	Integration

TC-10	Normal	Whitebox	Functional	Integration